

The Six Orthogonal Shifts - detailed version 2

A Framework for Deploying AI at Scale

Why six, and why these six?

A diffusion pathway has to answer two questions: *what are you deploying?* (the 30% — technology) and *how does it get absorbed?* (the 70% — everything else).

Within the 30% (technology), the independent dimensions are:

1. what you build **on** (foundations)
2. what you build **with** (choices)

Within the 70% (non-technology), the independent dimensions are:

3. who **transforms** (the institution)
4. who **executes** (the ecosystem)
5. who **absorbs** (the workforce)
6. what makes it **last** (the operating model)

These six are orthogonal — you can vary any one without automatically changing the others, and you can't collapse any two without losing a decision that an adopter has to make independently. Two sit in the 30% (technology). Four sit in the 70% (everything else).

The 30% breaks into two independent dimensions: **what you build on** (your data, your problem framing — the foundation that determines what the technology addresses) and **what you build with** (your models, vendors, architectural patterns and components — the parts you assemble and must be able to swap).

The 70% breaks into four: **who transforms** (the deploying organisation that must change how it works), **who executes** (the partners, vendors, and collaborators who do the work you shouldn't do yourself), **who absorbs** (the frontline people who must take the technology into their daily practice and stay more capable because of it), and **what makes it last** (the velocity, governance, and funding design that determines whether this runs for five years or dies after the pilot).

These six dimensions are the minimum. Remove any one and you've lost a question the adopter must answer independently. Add a seventh and you'll find it collapses into one of the existing six.

The Three Levels

Level	What it answers	Count
Level 1	What is the ratio?	30% tech / 70% non-tech

Level 2	What are the independent shifts?	6 shifts (2 tech + 4 non-tech)
Level 3	What are the specific decisions within each shift?	21 sub-components (7 tech + 14 non-tech)

Shift	Category	Sub-components
A. Problem orientation (what you build on)	30% tech	A1 Problem framing, A2 Data posture, A3 Existing assets, A4 Proof
B. Architecture (what you build with)	30% tech	B1 Model choice, B2 Data sovereignty, B3 Vendor independence, B4 DPG vs instance
C. Institution (who transforms)	70% non-tech	C1 Framing, C2 Resistance & non-transferability, C3 Institutional knowledge
D. Ecosystem (who executes)	70% non-tech	D1 Ecosystem design, D2 Trust source, D3 Coordination mechanism, D4 Network operator
E. Workforce (who absorbs)	70% non-tech	E1 Training timing, E2 Training depth, E3 Agency test
F. Operating model (what makes it last)	70% non-tech	F1 Velocity, F2 Governance, F3 Sustainability, F4 Pilot to deployment

The 7:14 ratio at Level 3 (33% tech, 67% non-tech) mirrors the 30/70 rule. The structure is fractal — the same pattern repeats at every level of resolution.

The 30%: Technology Shifts

The technology side has two independent dimensions. What you build on — your problem and data foundation. And what you build with — your models, components, and architectural choices.

Shift A: Problem orientation — what you build on

Question it answers: Where do I start?

FROM “Let’s start with the technology/data/app/bot” → **TO** “Let’s start with a specific problem”

A1. Problem framing

FROM “We want to use AI” → **TO** “We want to solve [specific problem] for [specific person]”

Not “we want to use AI.” A specific problem: children in grade five who cannot read, farmers who do not know tomorrow’s weather advisory, district officials who cannot see their own data. If you cannot name the person whose life gets better and explain how, stop here. The problem determines which data matters, which models fit, and what “good enough” looks like. Diffusion works from the use case inward, not the product outward.

Problem framing is not just the adopter's first task. It is the hardest task to extract from builders who have already deployed. In curating sixty-seven AI impact stories for the AI Commons, the team found that more than half the builders could not articulate their own problem statement. It had to be written for them, after interviews. This means the pathway's first function — before presenting solutions — must be helping the adopter formulate their problem. A problem well-stated is a pathway half-found.

Specificity matters. There is a gap between “we want to use AI in agriculture” and “leaf curling in chilli — what to do?” or “can guava be grown on my land?” The 205,000 crop/pest queries per month in MahaVISTAAR are evidence that when the problem is framed at the right granularity, adoption takes care of itself. The test: can you name the specific question the specific person is asking today?

The system's categories must also match the farmer's mental model, not the deployer's organisational chart. Grouping use cases by what farmers are *trying to do* (asking about a sick animal, checking scheme eligibility, tracking an application) produces a different — and more adoptable — architecture than grouping by technical taxonomy.

One further distinction: problems come in two modes. **Inbound** — the farmer comes with a question. **Outbound** — the system anticipates a need (pink bollworm detected in your region, heavy rainfall expected tomorrow) or seeks information the farmer has. Outbound requires the system to frame problems the farmer hasn't articulated yet, using crop calendar data and location signals. That is a different design task from answering a question someone already asked. The shift from “platform that waits for questions” to “platform that anticipates needs” is where advisory systems move from useful to indispensable.

A2. Data posture

FROM “Clean the data first, then deploy AI” → **TO** “Deploy first; AI creates the demand for connected, clean data”

The unlock is access, not quality. Large language models are designed to work with messy, unstructured, heterogeneous data from multiple sources. The data question should be reframed from “what data do you have and how good is it?” to “what data exists across your silos and can we connect it?” Once connected, AI reveals what the data actually contains, which creates the institutional demand for cleaning and standardisation. The demand follows the deployment, not the other way around.

In Maharashtra, once AI started pulling from databases that had never been queried together, officials asked “whose data is right?” for the first time. Departments had APIs nobody had ever called. The moment AI wired them together, officials were stunned by what their own data could show. AI does not just use data. It reveals what the data actually is. Waiting for perfect data before deploying is waiting for a reason that will never arrive on its own.

Data unlock — connecting existing databases that sit in silos — is a distinct concept from data readiness or data quality. Malawi has sixty-seven health databases, all unconnected. MahaVISTAAR pulled from APIs that had existed for years but had never been called. Nobody needed to clean them first. The connection itself was the intervention.

Data also flows in both directions. The system returns anonymised, aggregated signals *back* to institutions — which advisories are being accessed, which crops have insufficient guidance, which regions show stress patterns. Data is not just an input to the AI. The AI creates a feedback loop that improves institutional data over time. Institutions listening at population scale.

Two architectural decisions sit within data posture that adopters must resolve early. First: do you **federate** (connect at query time, each institution retains ownership, the AI accesses data at the moment of query) or **aggregate** (copy into a central warehouse)? Second: is data accessed on an opt-in, consent-based basis, or is it pre-loaded? Both are data posture questions with governance implications (see F2).

A3. Existing assets

FROM *“We need to build new infrastructure before we can start”* → **TO** *“Start with what you have — every department has a database”*

The pattern that kills adoption: someone says “you need a registry” and the response is “forget it, I can’t do any of this.” But every department has a database. It may not be a perfect registry, but it is a starting point. Nobody starts by running a marathon. You start with what you have — an existing database, a PDF textbook, a set of unconnected APIs — and train towards the goal. The CDPI functional framing shows how: even a simple textbook PDF can power contextual advice when combined with an AI agent and location data.

Before you build, map what exists. Maharashtra had fragmented advisory platforms, university databases, weather services, and market data — all working in isolation. The feasibility question was not “what do we need?” but “what can be joined?” Every deployment that moved fast started with an inventory, not a specification.

Asset readiness varies. Amul came with 2 billion procurement transactions, 50 years of cooperative data, 1,200+ veterinary doctor records. Ethiopia had limited digital advisory infrastructure. The same pathway worked in both, but the starting point — and therefore the first step — differed. The question for any adopter: on a readiness gradient, where does your existing asset base sit, and what is the minimum viable starting set?

A4. Proof

FROM *“Show benchmarks, conference demos, or vendor-specific results”* → **TO** *“Show it working in a real setting, with real users, solving a real problem — and make the proof product-independent”*

There is no shortage of money, technology, government willingness, or private sector innovation. The only thing missing is proof. Working demonstrations in real settings, with real users, solving real problems. Not benchmarks. Not conference demos. And the proof must be product-independent — the moment you tie it to one vendor’s platform, you have proved that their product works, not that the approach works.

Proof operates at two levels, and both are necessary. The first is **demonstration proof** — showing a sceptical leader that AI works on a real problem. Demonstration proof triggers the decision to start. The

second is **experiential proof** — the leader having deployed it themselves and emerged with a different understanding of their own organisation. The Amul CEO's conversion — from "data is the new oil, I don't understand what that means" to "after I saw data coming together, I started seeing the magic" — happened because he saw AI working on his own data, with his own operations. A slide deck could not have produced that conversion. Demonstration proof triggers the decision to start. Experiential proof creates the champion who can sell it internally and to others.

The most powerful proof is the **compression sequence** itself. Nine months (Maharashtra, pioneer). Three months (Ethiopia, first international adopter). Three weeks (Amul, cooperative with existing data infrastructure). The proof is not "the system works." The proof is "the pathway makes the next adoption faster." That is a different evidentiary claim, and a stronger one.

Proof from five different institutional types — a state government department, a national ministry, a cooperative, a state government with an independent build, and a national transformation institute in a different country — also demonstrates that the pathway is not context-specific. Variety of proof contexts is itself a form of proof.

Shift B: Architecture and components — what you build with

Question it answers: What technology choices keep me flexible as everything keeps changing?

FROM *"We need the best model / best data / one vendor"* → **TO** *"We need an architecture that lets us evolve and swap any component"*

B1. Model choice

FROM *"We need the best / latest AI model"* → **TO** *"Normal AI is good enough; models are pluggable buttons on the shirt"*

The frontier model race is irrelevant to most of the world's problems. The AI that exists today is adequate for agriculture advisories, health screenings, educational support, government service delivery. The gap is not in model capability. It is in institutional readiness to use what is already available. Waiting for a better model is an excuse, not a strategy. If your people cannot say "show me the algorithm," you have not deployed AI. You have outsourced your judgment.

Model choice involves more than picking a name. The architecture needs an independent moderation layer — a separate model that performs domain validation, content safety filtering, and prompt injection defence — decoupled from the advisory engine. You don't choose one model. You choose at least two (one to moderate, one to advise), and they must be independent.

Model evaluation is as important as model selection. Adversarial test sets of up to 500 attack patterns and domain-specific accuracy benchmarks determine whether a model is working, not just whether it was the right pick at procurement time. Fine-tuned smaller open-source models have achieved 94% accuracy against 91% for larger commercial APIs on field evaluation sets. The deployment topology

question — commercial API (variable cost, lower control) versus self-hosted (fixed cost, full control) — carries cost, latency, and sovereignty implications that compound over time.

B2. Data sovereignty

FROM *“Send data to the model provider for processing”* → **TO** *“Data stays with the deployer; only the prompt goes to the LLM”*

The orchestrating agent talks directly to the deployer’s databases, pulls the data, frames the prompt, and sends only the prompt to the LLM. The raw data never leaves. This is a sovereignty decision, not a technical footnote. Prompt-level interaction protects data better than contracts do. If a vendor’s architecture requires that your data be sent to their servers, they have designed a system that works for them, not for you. Data must create a story of access, not extraction — the farmer’s data should provide intelligence back to the farmer.

Operationally, this means the AI connects to data sources as tools at the moment of query. External data sources and DPI foundations are accessed on demand, not pre-loaded into the model. Sovereignty is not just about where data resides — it is about *when* the AI accesses it.

Every claim in the system’s response should be grounded in a verified source with attribution. The farmer does not receive an AI opinion — they receive institutional knowledge, delivered through an AI interface, with the source named. The institution’s authorship stays visible in every answer.

B3. Vendor independence

FROM *“One vendor who can do it all”* → **TO** *“Open standards, open protocols; build for the swap”*

Open source, open protocols, open standards — these are survival strategies, not ideological preferences. If you build on one vendor’s proprietary stack, you have given that vendor control over your capability. Vendor capture happens when large technology companies have more sales engineers than most government departments have technologists. The information asymmetry is severe. The question after every pilot: who controls what scales? If the answer is one vendor, you have a dependency, not a deployment. Product-independent proof is the only proof that enables diffusion.

At the protocol level, open standards like Beckn allow standardised discovery and interaction across independent providers — vendor independence enforced through protocol design, not just contractual terms. The architecture should be designed so that each layer can evolve independently. The AI model can be swapped without changing the data sources. The voice pipeline can be upgraded without changing the orchestration logic. No AI model is permanent. The infrastructure beneath remains stable as models evolve.

B4. DPG versus instance

FROM *“Building and deploying are the same thing”* → **TO** *“Separate the reusable code (the DPG) from the deployment instance”*

Separate what you build (the reusable code, the digital public good) from where you deploy it (the instance — Maharashtra’s infrastructure, Amul’s app, Ethiopia’s systems). The code sits in one repository. The deployment sits in the deployer’s infrastructure with the deployer’s data. The code builder’s

accountability stops at the code. The deployer's accountability starts at the deployment. When this line blurs — when people confuse building with deploying — the whole system stalls on questions of liability and accountability that have nothing to do with technology. Drawing that line clearly is not a technical task. It is an institutional one.

The DPG is not just code — it is code plus documented organisational knowledge (a collaboration blueprint covering governance models, enabler networks, and institutional arrangements). The instance is not just a deployment — it is an adaptation that modifies both. A DPG that insists on being deployed as a full stack is behaving like a vendor, not a public good. Reuse at the component level — not full stack — is what drives down the cost and time of each successive deployment.

Ethiopia's three-month deployment was possible because the architecture, governance frameworks, language pipeline methodology, data connector approach, model evaluation benchmarks, and failure mode library were all transferable assets. The question for any DPG: what is the specific list of assets that transfers from the public good to the next instance?

The 70%: Beyond-Technology Shifts

The non-technology side has four independent dimensions, each tied to a different actor or design question. Who transforms — the deploying institution. Who executes — the ecosystem of partners. Who absorbs — the frontline workforce. And what makes it last — the operating model.

Shift C: Institutional transformation — who transforms

Question it answers: What kind of undertaking is this?

FROM *"This is a technology project"* → **TO** *"This is an institutional change project that uses technology"*

The adopting institution often does not know what is possible. This is not resistance — it is absence of imagination. Governments that have never seen AI working on their own data, in their own language, for their own problems, cannot formulate a vision of what they want to do. They are not saying "no." They are saying "I don't know what to say yes to."

C1. Framing the undertaking

FROM *"This is a technology project — hire engineers, buy a model"* → **TO** *"This is an institutional change project; 70% is transformation work you already know how to do"*

If you frame this as a tech project, you will buy a model, run a pilot, write a report, and move on. The reframe that makes this less frightening: the 70% that is non-tech — institutional structures, policy, funding, capacity building — is familiar ground for any government that has done a transformation project before. It has nothing to do with AI. The diagnostic is not that your heart is failing; it is that what you are eating is causing the problem. Change the lifestyle, not the organ.

Different institutional actors need different framings. A system declarant (Chief Secretary, PMO) needs vision and possibility — what could this do for the people I serve? An evaluator (department head) needs feasibility and options — can I do this, and what are the trade-offs? A constructor (CTO, programme lead) needs specifications. An agent of change (extension officer, district official) needs operational guidance. The pathway must frame the undertaking differently for each.

Simulation, not documentation, is how institutional imagination is activated. Not workshops where experts lecture. Not case studies where other countries' experiences are described. Simulated experiences where the adopting institution's own people encounter decision points and trade-offs in a compressed timeframe, using their own data and context.

Bounded ambition is also a framing strategy. Starting with one or two crops, a few districts, one or two languages produced the early proof that enabled scaling. You frame the undertaking not as “transform state agriculture” but as “run a bounded pilot in Kharif crops in selected districts.” The institutional appetite for the latter is fundamentally different from the appetite for the former.

C2. Institutional resistance and non-transferability

FROM *“Officials who resist are obstructing progress”* → **TO** *“Resistance is rational; and past success at one thing does not create readiness for the next”*

This sub-component has two legs, both about why adoption stalls — but for different reasons.

The first is **resistance**. Bureaucratic inertia is not stupidity. Officials who resist AI adoption are often protecting something real: discretion, authority, familiar processes that work well enough. Automated systems remove the ability to grant or deny on a case-by-case basis. For officials whose authority comes from that discretion, AI is a threat. This resistance is rational, and pretending otherwise is naive. But it must be overcome, because the same inertia that protects stability also prevents improvement.

One way to dissolve resistance: reframe the problem from “officials blocking AI” to “the system failing to reach people.” When you show an agriculture secretary that their own scheme portals are structurally inaccessible to the farmers they were built for — that a determined person would need a smartphone, reliable internet, literacy, time, and the right language to access what the department already holds — resistance often shifts. The problem is not AI. The problem is that the department's own knowledge doesn't reach the people it was meant for. AI is the solution, not the threat.

The second is **non-transferability**. Knowing how to do one thing does not mean knowing how to do the next thing. Past success creates confidence but not readiness — these are different things. Each new technology requires its own cycle of learning, trust-building, and institutional adaptation. This takes time and cannot be skipped.

Non-transferability also operates across institutional types. Bihar Krishi was built independently, with different partners, under a different institutional logic. A cooperative learns differently from a state department, which learns differently from a national ministry. The pathway each walks will differ even when the framework is the same.

The first leg requires narrative reframing. The second requires a new learning cycle. Both are reasons the adopter's institution is not ready, but the interventions are different.

C3. Institutional knowledge

FROM *“Our expert knows how this works”* → **TO** *“If it lives in one person’s head, the institution has learned nothing”*

AI forces institutions to articulate what they know, what they decide, and why. Most institutions have never had to do this because the knowledge lived in individuals and documents. When those individuals transferred, the knowledge went with them.

There is a specific leadership act involved: authorise your institution’s knowledge to be made accessible — and stand behind what it says. When the Department of Agriculture, Maharashtra, authorised its advisory corpus to flow through MahaVISTAAR, it made an institutional knowledge claim: *this is what we know, and we certify it*. That is the act of institutionalising knowledge by making it publicly queryable.

AI also reveals what knowledge the institution *lacks*. 205,000 crop/pest queries per month create a visible, quantified demand signal for knowledge that was previously tacit. Where farmers’ questions go unanswered, the institution now sees its own gaps.

Shift D: Ecosystem and trust — who executes

Question it answers: Who does what, how do we work together — and where can AI itself reduce the coordination cost?

FROM *“We need to build everything ourselves / Trust will come once people see results”* → **TO** *“Do the minimum; let the ecosystem do the rest — and use AI itself to enable coordination even before full consensus exists”*

When multiple actors from different countries, sectors, and institutional contexts all decompose their experiences against the same six-shift framework, they create a shared language without having to agree on standards, formats, or metrics. A Kenyan agricultural agency and a Solomon Islands fisheries department can find relevant pathway components from each other’s work — not because their contexts are similar, but because the decomposition framework makes the transferable elements visible.

D1. Ecosystem design

FROM *“We need to build everything ourselves”* → **TO** *“Do the minimum; let the ecosystem do the rest — you are one part among many”*

MahaVISTAAR required 25+ organisations across government departments, model providers, voice partners, funders, data providers, and domain experts. This biryani is not rice and sambar. It has these many ingredients, and they are all necessary — not sufficient, but necessary. But the complexity can be made simple: there are organisations giving paid services, organisations giving free services, and the deploying government. What are you providing, and who is accountable if it fails?

A four-layer taxonomy helps an adopter inventory their own ecosystem: institutional and governance (funders, orchestrators, government bodies), technology and AI (R&D, language models, knowledge contributors), structured data (live feeds), and knowledge and documents (guidelines, publications, research). MahaVISTAAR mapped 54 enablers across these four layers. The composition varies — Maharashtra required 25+ organisations, Amul required fewer because it brought its own cooperative infrastructure, Ethiopia required different partners entirely — but the four-layer structure holds constant.

The pathway ecosystem also requires an incentive structure. Builders who have deployed AI contribute their decomposed experience — not as a case study, but as answers to the six-shift questions, updated periodically with operational data. In exchange, they get access to the full knowledge base, visibility at global summits, and early access to funding opportunities. Those who do not contribute lose access. The currency is knowledge, not money. Dead ends must also be managed: when a contributor's deployment stops working, that node must be flagged — not deleted, because the failure itself is valuable knowledge, but marked so that future adopters are routed around it with appropriate warnings.

D2. Trust source

FROM *“Trust will come once people see the results”* → **TO** *“Trust has a source — who delivers the advice matters as much as what it says”*

Trust cannot be bought. In one project, the conversations alone took 18 months before serious work could begin. There are no shortcuts. But the deeper insight from the field: a company with 25 lakh farmers can have all the right information and still not be trusted — because the farmer distinguishes between information and trusted information. The same advice from a commercial seller sounds like a sales pitch; from a university or government extension service, it sits as certain. Water does not become sacred water just because it comes from the same well — it has to pass through the right source. YouTube gives information, but it sits tentatively. Expert knowledge from a trusted source sits as certain.

Operationally, the AI must not speak as itself — it must speak as the institution. MahaVISTAAR cites the Department of Agriculture and state universities. Sarlaben is named to signal that it's built for the woman producer. Every response carries institutional attribution. The AI's identity must be subordinated to the institution's identity.

Voice also carries trust in a way text often does not. A Marathi-speaking farmer hearing advice in Marathi, from a number associated with the state agriculture department, trusts it differently from the same advice delivered as English text on a website. Trust source and channel choice are deeply connected at the farmer's end, even when they are separable at the design level.

D3. Coordination mechanism

FROM *“Get everyone to agree on standards, formats, and metrics before starting”* → **TO** *“Shared language first; AI can enable coordination even before full consensus exists”*

The traditional approach requires all stakeholders to agree on data formats, metrics, governance structures, and procedures before work begins. That is what creates the 18-month pre-work timeline. AI offers a different mechanism: it can work with unstructured, heterogeneous inputs from multiple actors

and create shared representations without requiring everyone to first agree. You can start working together before you've achieved full consensus, and let consensus emerge from the collaboration rather than being a prerequisite for it. But this requires a shared language — not shared metrics or shared formats, but a common way of seeing the problem. The emphasis is not on the language. It is on the *shared*.

At the protocol level, open standards like Beckn provide a coordination substrate: standardised discovery and interaction across independent providers. You don't need everyone to agree on data formats. You need a shared protocol that allows systems to discover and interact with each other. The Bharat-VISTAAR model — a national layer with MahaVISTAAR, Amul, Bihar Krishi, and Ethiopia as nodes — shows hub-and-spoke federation in practice. Each node retains local specificity; the national layer provides shared knowledge base, scheme connectivity, and integration.

D4. Network operator

FROM *"It's everyone's responsibility, which means it's nobody's"* → **TO** *"Someone must hold the network together and say 'this runs in my name'"*

An ecosystem without a committed network operator is a coalition without a spine. What is paramount is the presence of someone who takes the risk, who says "I call end of it — this runs in my name." That entity is not a brand. It is the entity that creates the network roots. Without it, partners default to either overreach or blame. In one project, the same institution that accused its operational partner of rubber-stamping vendor decisions also refused to participate in the steering committee that was supposed to govern those decisions. Accusation and abdication from the same source. The hunter and the farmer have different roles. The mistake is asking farmers to hunt and hunters to farm.

The network operator role has been played by different institutional types across deployments: a state government department (Maharashtra), a national ministry (Bharat-VISTAAR), a cooperative (Amul), a national transformation institute (Ethiopia/ATI). Any entity that can say "this runs in my name" and has the institutional standing to back that claim can play this role. What matters is not the type of entity but the commitment.

Shift E: Workforce and capability — who absorbs

Question it answers: How do my people absorb this — and stay above the algorithm?

FROM *"We'll train people once the system is ready"* → **TO** *"Training IS the deployment — and must leave people more capable, not more dependent"*

E1. Training timing

FROM *"We'll train people once the system is ready"* → **TO** *"Training IS the deployment — not a phase after it"*

What translates technology into actual change is the know-how that allows people to absorb it and apply it in their own context. A fifth-grader who cannot read does not need better content. She needs practice, delivered in a way that builds confidence. That is a training design problem, not a technology problem. You cannot bolt training on at the end.

For end users, the best training is use itself. Farmers weren't trained on MahaVISTAAR and then given access. They called a number, asked a question, and got an answer. The natural progression — from simple queries (weather, pest) to complex actions (scheme applications, credit products, grievance tracking) — means each step trains the farmer for the next. Design the system so that use *is* training.

For institutional workforces, training is different — structured, deliberate, scaled. Bihar Krishi trained 15,000+ extension workers across 38 districts. Training timing for the beneficiary and training timing for the institutional workforce are different design problems that require different solutions.

E2. Training depth

FROM *“Train the frontline users”* → **TO** *“Who trains the trainers? If knowledge stops at the first layer, it won't survive the second posting cycle”*

Your rollout plan probably has a line item for “training.” But who trains the people who train others? How deep does the understanding go? Can a district official explain to a sceptical colleague why the AI recommended what it did? If knowledge stops at the first layer, it will not survive the second posting cycle.

The institutional challenge is not getting people to use the system. It is building enough understanding that the system survives the departure of every individual who helped build it. When extension officers rotate — a routine event in Indian government — does the institutional capability go with them? The test: can the system function when every individual who helped build it has moved to a different role?

E3. Agency test

FROM *“AI replaces human effort with better answers”* → **TO** *“AI must leave people more capable, not dependent — informed, therefore powerful, therefore nobody can fool me”*

Does the farmer become a better farmer, or merely a better button-presser? If an extension officer stops exercising professional discretion because the AI provides the answer, does that hollow out the very institutional capability the pathway depends on?

The Amul deployment surfaced both sides. Younger family members who lack thirty years of dairy experience can now access expert knowledge through Sarlaben — “informed, therefore powerful to make choices; freedom comes from knowing, and nobody can fool me.” That is the right outcome: AI as an equaliser that gives the less-experienced person access to what the expert knows, while preserving their autonomy.

But the watchword from the same field: information alone is not the point. There is skill in the hand that counts for more than information in the head. AI should make the farmer's skill more precise and their choices more informed. It should not replace the skill itself. If your pathway produces people who can't

function when the AI fails, you've built a system that works until it doesn't. The test: after twelve months on the pathway, can the farmer still function if the AI goes down?

Agency also has a progression. Farmers in the field have moved from information queries to transactional actions — applying for schemes, tracking grievances, exploring credit. From passive information recipient to active agent using the system to get things done. The pathway should explicitly design for this progression.

A well-designed simulation serves double duty: training mechanism and proof. When an adopter runs through a simulated deployment, they should emerge with genuine understanding of the trade-offs they face, the decisions they must make, and the reasons behind those decisions. The simulation builds the adopter's capacity to make independent judgments when the AI fails, when the context changes, or when a new problem arises that the pathway did not anticipate.

Shift F: Operating model — what makes it last

Question it answers: What lets me move fast, stay safe, and keep this running after the builders leave?

FROM *"Let's do a big launch / Governance slows things down / We'll figure out sustainability later"* →

TO *"20 pilots in 20 weeks / Governance is what lets you go fast without breaking things / If the funding model is unclear, the project is already dead"*

F1. Velocity

FROM *"Plan thoroughly, then do a big launch"* → **TO** *"20 pilots in 20 weeks; a series of plus-ones, not plus-ten"*

Speed and scale are the best insurance against the negative coalition that will inevitably develop to stop anything new. Machiavelli said it in 1532: the reformer has enemies in all those who profit from the old order. The best insurance is visible results before the opposition organises. Within a day of someone approaching you, send them a package. Within a week, they should be running a pilot. Within three months, you should know if it works.

The Amul pattern shows how: start with Sarlaben advisory, then add AI-based booking for artificial insemination — a small plus-one on what already exists. Then microcredit via the cooperative bank. Then Bharat Taxi via open network protocols. Each step builds on existing readiness. Never plus-ten.

Velocity also operates across deployments, not just within them. The entire pathway compresses with each successive adoption. Nine months, then three months, then three weeks. Velocity is not just about moving fast within one deployment — it is about making each successive deployment faster by transferring assets and failure-mode knowledge.

Proactive push is a velocity accelerator at the user level. MahaVISTAAR's proactive voice alerts reach 17 lakh farmers daily. Instead of waiting for farmers to discover the system, the system reaches out. The design question: how quickly can you shift from pull (farmer calls you) to push (you call the farmer)?

F2. Governance

FROM *“Governance and regulation slow things down”* → **TO** *“Governance is the condition for sustainable speed; design it first, then go fast — and fix procurement while you’re at it”*

Governance is not the enemy of speed. It is what lets you scale without stopping to fix things that should have been right from the start. Policy for AI, data protection standards, regulatory compliance, quality benchmarks — the model where access is universal and defaulters are caught through analytics is the opposite of models that put high entry barriers to keep everyone out. Going fast without governance is going fast towards a wall. The same officials who share sensitive data on WhatsApp will cite privacy concerns to block a properly governed AI system. The solution is better governance, not no deployment.

Governance includes procurement, and procurement is where many deployments die before they start. Government procurement was built for buying hardware and hiring body shops. It was not designed for buying AI capabilities that evolve monthly. RFP specifications become obsolete before the contract is signed. The process itself prevents adoption of the technology it is trying to procure. Seventy percent of government respondents say digitalization takes four years from idea to first beneficiary benefit. Much of that delay sits in procurement. An adopter can design good data protection, good quality benchmarks, and a good regulatory posture and still be blocked by a procurement cycle that takes six months to authorise a model that will be two generations old by the time the contract is executed. Procurement reform is a governance dimension, not a separate problem.

Governance of AI safety is also an architectural concern, not just a policy one. An independent moderation layer — 500 adversarial attack patterns, domain validation, prompt injection defence — ensures unsafe outputs are caught before they reach the farmer, not just addressed by policy after the fact.

F3. Sustainability

FROM *“We’ll figure out who pays and who maintains it later”* → **TO** *“If the funding model is unclear, the project is already dead”*

Who maintains the AI system in year three? Who pays for inference costs when the grant runs out? Who decides when the model needs retraining? Who trains the trainers for the next posting cycle?

Technology evolution outpaces institutional adaptation. A model you train today may be obsolete in six months. Total cost of ownership for AI at population scale is two to five times the number in the original proposal. If you have not answered these questions before you start, you have built a demo that will be switched off the moment the project champion moves to a different posting.

The sustainability question now has concrete numbers. A pathway costs approximately \$250,000 to set up and \$250,000 per year to maintain. The maintenance cost breaks down into model inference costs (which drop as you switch to open-source and cache common queries), telephony and data transfer costs (which are operational and relatively fixed), and the enterprise service cost (which drops as DPG components replace custom-built modules).

The cost curve bends downward with each successive deployment because shared components — evaluation benchmarks, voice data models, orchestration frameworks — become public

goods that the next deployer inherits for free. The financing model for one hundred pathways is not “one hundred times \$250,000.” It is a declining marginal cost curve where the first twenty pathways cost roughly \$5 million total, and the next twenty cost significantly less because the reusable infrastructure is already in place.

National-layer amortisation also shifts the economics. Bharat-VISTAAR’s Rs. 150 crore allocation creates a national layer that amortises infrastructure costs across all state nodes. Maharashtra does not bear the full cost of the knowledge base, AgriStack, or ICAR integration — those sit at the national level. Build once at the national level, deploy many times at the state and cooperative level.

F4. Pilot to deployment

FROM “The pilot worked — now scale it up” → **TO** “The pilot is not the point; design for steady state from day one”

Everyone can do a pilot. A pilot has project funding, excited volunteers, and weekly check-ins. A deployment has none of those. Who runs this at steady state? Who handles bugs on a Tuesday morning? Who decides when the model needs retraining? If the operating model is “same as the pilot but bigger,” it will fail. No widely accepted template for “how to run AI in government” exists. Every organisation invents its own, makes its own mistakes, learns its own lessons. The knowledge does not transfer because the institutional mechanisms for transfer do not exist. That is the gap the diffusion pathway must fill.

The pilot’s output is not a working system — it is a set of learnings that inform the deployment design. An eight-week pilot followed by structured review, before any decision on scaling. Those who treat the pilot as a vanity launch find the compounding stops there.

The steady-state operations question must be answered before the pilot plan is written. Who handles bugs on a Tuesday morning? Who decides when the model needs retraining? Write the operating manual for year three before you write the pilot plan for month one.

Two Open Questions

1. Inclusion architecture does not have a clean home in the framework

The strongest conceptual move in recent field work is naming the inclusion problem — “the system is failing to reach them” — and positioning voice-first as the architectural response. Voice is not a model choice (B1). It is not a data sovereignty decision (B2). It is not vendor independence (B3). It is not DPG vs. instance (B4). It is an independent architectural commitment: the system must work on a basic feature phone through a voice call, and this requirement shapes the entire technology stack — ASR, TTS, turn detection, telephony gateway hardware, latency budgets.

This sits at the intersection of A1 (problem framing), B (architecture), and D2 (trust source). In a framework built on orthogonality, that intersection is a stress point. It may belong as an A-level

sub-component: A5 — Inclusion design. FROM “build for smartphone users and hope others catch up” TO “build for the person with the least access and let that design serve everyone.” The test for orthogonality: can you vary your inclusion design independently of A1-A4? You could frame the right problem, have the right data posture, start with existing assets, and have proof, but still build for the wrong user.

2. The seven-layer system architecture is richer than Shift B captures

The component enumeration from field deployments (user layer, interface layer, moderation layer, AI decision engine, knowledge/scientific models, live data sources, DPI foundation) describes an architecture that is more granular than B1-B4. The framework treats architecture as four high-level decisions (model, data sovereignty, vendor independence, DPG/instance). Field experience reveals that within those four decisions, there are at least seven separable layers, each with its own sub-decisions. This does not break the framework — B1-B4 are the right Level 3 categories. But it means that the depth beneath Shift B is greater than beneath other shifts. An evaluator will need architectural guidance at a level of detail that B1-B4 alone cannot provide.